

Process

Pratheeba Jeyanathan

Faculty of Engineering
University of Jaffna

Lecture plan – Chapter 2

- Lecture hours: 12 hours
- Teaching method: Powerpoint slide and conventional teaching method
- Assignments: 9 hours
- Practical: 9 hours
- References: 1. Silberschatz, P. B. Galvin, and G. Gagne, “Operating System Concepts”, 9th Edition, 2013 –Chapter 5 2. S. Tanenbaum and H. Bos, “Modern Operating Systems”, 4th Edition, 2014 –Chapter 2
- Acknowledgment: Slides are from [1] & [2] and modified.

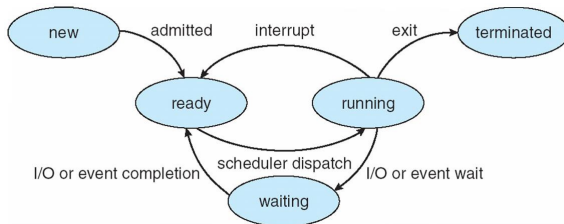
What is process?

- A program in execution is called as a **process**. Execution of the process must be in a sequential fashion.
- Terms *jobs* and *process* are interchangeable.
- Parts of a process:
 - **Text section** of the program called as **program code**.
 - Current activity including program counter, process registers.
 - Stack containing temporary data such as function parameters, return addresses and local variables.
 - Data section containing global variables.
 - Heap containing memory dynamically allocated during run time.
- Process is an active entity, while program is passive stored on a disk. When the executable file is loaded into the memory, the program becomes process.
- A GUI mouse click or command line entry of program name can start the execution of a program.
- In case of multiple users executing the same program, a single program can be several processes.

States of a process

- During the execution, a process changes it's state.
 - new: the process is being created.
 - running: instructions are being executed.
 - waiting: process wait for some event to occur.
 - ready: process wait to be assigned to a processor.
 - terminator: a process which has finished it's execution.

Process state diagram:



Process Control Block (PCB)

- Information related to each process are:
 - **Process state:** State of the process
 - **Program counter:** location of the next instruction to be executed
 - **CPU registers:** Contents of all process-centric registers
 - **CPU scheduling information:** Such as priorities, scheduling queue pointers
 - **Memory management information:** Memory allocated to the process
 - **Accounting information:** Such as CPU used, time limits and clock time elapsed since start
 - **I/O status information:** Such as I/O devices allocated to process and list of open files.

Process scheduling

- This is to maximize the CPU use – quickly switch processes onto CPU for time sharing.
- Process scheduler is used here to select one process from available processes for next execution on CPU.
- Scheduling queues are maintained:
 - **Job queue:** This contains the set of all processes in the system.
 - **Ready queue:** Set of all processes residing in main memory, ready and waiting for the execution.
 - **Device queue:** This queue contains a set of processes waiting for some I/O devices.
- Processes move among the various queues.

Schedulers

- **Short-term scheduler (CPU scheduler):** It selects the process to be executed next and allocate it to the CPU. Sometimes this is the only scheduler in the system. This must be fast as it invoked frequently (milliseconds).
- **Long-term scheduler (job scheduler):** Selects the process to be brought into the ready queue. This might be slow as it invoked infrequently (seconds, minutes).
- Two types of processes are:
 - **I/O-bound process:** Spends more time on I/O operations than computations. Has many short CPU bursts.
 - **CPU-bound process:** More time on doing computations. Few very long CPU bursts.
- **Medium-term scheduler:** This can be added to the system, if the degree of multiple programming needs to be decreased – removes process from memory, store it on disk, bring back from disk to continue execution, which is called as swapping.

Context switch

- While switching the CPU to another process, this context switching is used to save the state of the old process and load the saved state of the new process.
- Context of a process is represented in the PCB.
- As the system does not do any useful work while switching, the context-switching time is an overhead. The more complex OS will lead to the longer context switch.
- Some hardware provides multiple sets of registers per CPU – multiple contexts loaded at once.

Process creation

- Parent process creates children processes, which create other processes. This will form a tree of processes.
- Process identifier (pid) is used to identify and manage processes.
- Resource sharing options:
 - Parent and children shares all resources
 - Children share subset of parent's resource
 - Parent and child share no resources
- Execution option:
 - Concurrent execution of parent and children
 - Parents wait until child terminates
- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples:
 - `fork()` - creates new process
 - `exec()` - call after `fork()` to replace the process' memory space with a new program.

Process Termination

- Process executes the last statement and then ask the OS to delete it using `exit()` system call.
 - Return status from child to parent via `wait()`.
 - Resources of that process are deallocated by the OS.
- `abort()` is a system call used by the parent process to terminate the execution of children process.

Some reasons for this abortion:

- Child has exceeded the allocated resources.
 - Task assigned to child is no longer required.
 - The parent is exiting – OS does not allow a child to continue if its parent terminates.
- If a process terminates, then all its children must also be terminated. The termination is initiated by the operating system.
- In cascade termination, all children, grandchildren, etc are terminated.
- `wait()` - parent process to wait for the termination of the child process. Returns the status and pid of the terminated process.
- If no waiting parent - zombie process. If the parent terminated without calling `wait` - orphan process.

Interprocess Communication

- A system can have independent or cooperating process.
- Other processes can affect these cooperating processes – including data sharing.
- Why do we need these cooperating processes?
 - Information sharing
 - Speedup the computation
 - Modularity
 - Convenience
- They need interprocess communication (IPC)
- Two models of IPC:
 - Shared memory
 - Message passing

Cooperating processes

- Cooperating processes are not like independent process. They can affect or be affected by the execution of another process.
- Advantages of process cooperation:
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

Producer-Consumer problem

- Here, producer process produces information which is consumed by a consumer.
- Two different buffers:
 - **unbounded-buffer:** No practical limit for buffer size.
 - **bounded-buffer:** Fixed size buffer.
- **Bounded-buffer – shared memory solution:** Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in=0;
int out=0;
```

This solution is correct with limited number of elements (BUFFER_SIZE-1).

Bounded-buffer – Producer

```
item next_produced;
while(true){
    /*Produce an item in next produced */
    while(((in+1)%BUFFER_SIZE)==out)
        /* do nothing */
    buffer[in]=next_produced;
    in=(in+1)%BUFFER_SIZE;
}
```

Bounded buffer – Consumer

```
item next_consumed;
while(true) {
    while(in==out)
        /*do nothing*/
    next_consumed=buffer[out];
    out=(out+1)%BUFFER_SIZE;
    /*consume the item in next consumed*/
}
```

Interprocess communication

Shared memory:

- An area of memory is shared within a set of processes that wish to communicate.
- User processes should control the communication, not the OS.
- The problem here is, a mechanism should be provided to the user processes to synchronize their actions while accessing the shared memory.

Message passing:

- This is another mechanism for process communication and synchronization.
- Here, processes communicate with each other without resorting to shared variables.
- Here *send(message)* and *receive(message)* are the two operations provided by IPC facility.
- Message size can be either fixed or variable.

Message passing ...

- If two processes want to communicate then,
 - they should establish a **communication link** between them.
 - message exchange via send/receive messages.
- Issues in this implementation:
 - How to establish the links?
 - Can a link be associated with more than two processes?
 - Number of links can be there between every pair of communicating processes?
 - Capacity of the link
 - Is the size of a message that the link can accommodate fixed or variable?
 - Direction of the link – unidirectional or bi-directional?
- Communication link implementation:
 - **Physical:** Shared memory, Hardware bus, Network
 - **Logical:** Direct or indirect, Synchronous or asynchronous, Automatic or explicit buffering

Direct communication

- Processes must name each other explicitly:
 - **send**($P, message$): Send a message to P.
 - **receive**($Q, message$): Receive a message from Q.
- Properties of a communication link:
 - Links are established automatically.
 - A link is between exactly single pair of communicating processes.
 - Exactly one link between each pair.
 - Usually bi-directional link, however it may be unidirectional.

Indirect communication

- Messages are managed and received from mailboxes (or ports).
 - Unique id for each mailbox.
 - Process can communicate only if they share a mailbox.
- Properties of communication link:
 - Link established only if processes share a common mailbox.
 - Single link may be associated with multiple processes.
 - Each process pair can share several communication links.
 - Unidirectional or bi-directional links.
- Associated operations:
 - Creation of new mailbox (port)
 - Send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - **send**(*A,message*): Send a message to mailbox A
 - **receive**(*A,message*): Receive a message from mailbox A.

Indirect communication ...

- P_1 , P_2 and P_3 share mailbox A.
- P_1 sends; P_2 and P_3 receive
- Now who gets the message?
- **Solutions:**
 - Allow a link to be associated with at most two processes.
 - Allow only one process at a time to execute a receive operation.
 - Allow the system to select receiver arbitrarily. Receiver should be notified by the sender.

Synchronization

- Message passing may be either blocking or non-blocking.
- Blocking is considered as synchronous.
 - **Blocking send:** Blocked sender till receive the message.
 - **Blocking receive:** Blocked receiver till the message is available.
- Non-blocking is considered as asynchronous.
 - **Non-blocking send:** Sender sends the message and continue.
 - **Non-blocking receive:** sender can receive a valid or null message.
- Different combination also possible – if both send and receive are blocking, we have a rendezvous.

Synchronization ...

Producer-consumer becomes trivial:

```
message next_produced;  
while(true){  
    /*produce an item in next produced*/  
    send(next_produced)  
}
```

```
message next_consumed;  
while(true){  
    receive(next_consumed);  
    /*consume the item in next consumed*/  
}
```

Buffering

- Queue of messages attached to the link
- Implemented in one of the following three ways:
 - Zero capacity: No messages are queued on a link – sender must wait for receiver (rendezvous).
 - Bounded capacity: Finite length of n messages – sender must wait if link is full.
 - Unbounded capacity: Infinite length – sender never waits.